

Segmentación de Núcleos en Imágenes de Citología Cervical para el Cribado Automatizado del Cáncer de Cuello Uterino

Ana Sofía López Romero, Andrea Nathaly Ochoa Torres, Viviana Alejandre Paniagua, Laisha Nicole Cortés Montes

Procesamiento de Imágenes Médicas · Grupo 601 · Tecnológico de Monterrey, Campus Guadalajara · Junio de 2026

Resumen—El cáncer de cuello uterino causó aproximadamente 342 000 muertes en 2020, siendo en gran medida prevenible mediante cribado citológico. Este trabajo implementa y evalúa un *pipeline* clásico de procesamiento de imágenes de siete etapas para la segmentación automatizada de núcleos celulares en citología cervical. El *pipeline* integra filtrado Gaussiano, CLAHE, umbralización adaptativa y operaciones morfológicas de apertura y cierre. Se evalúa sobre SIPaKMeD (4 049 imágenes de portaobjeto completo) y Cx22 (1 320 imágenes de célula individual, 512×512 px), lo que permite separar el efecto del método del efecto del formato de imagen. Sobre SIPaKMeD, la Precisión de píxel alcanzó 0.899 (≥ 0.88 ✓); el IoU medio (0.125) quedó por debajo del umbral por desbalance extremo de clases (núcleo $< 1\%$ del área). Sobre Cx22, el mismo *pipeline* sin re-ajuste obtuvo $\text{IoU} = 0.626 \pm 0.091$ y $\text{Dice} = 0.728 \pm 0.102$, un incremento de +0.501 en IoU respecto a SIPaKMeD, confirmando que el bajo rendimiento previo era un artefacto del formato de imagen y no del método. El *pipeline* es interpretable, determinista, no requiere GPU y procesa una imagen en < 12 ms en CPU estándar. Código disponible en github.com/laishacormon-dev.

Index Terms—Citología cervical, segmentación de núcleos, CLAHE, umbralización adaptativa, morfología matemática, SIPaKMeD, Cx22, IoU, Papanicolaou, cáncer cervical, VPH.

I. Introducción

El cáncer de cuello uterino es causado principalmente por cepas de alto riesgo del VPH-16/18. En 2020 se registraron 604 000 nuevos casos y 342 000 muertes [1], siendo la cuarta neoplasia más frecuente en mujeres. La prueba de Papanicolaou detecta cambios displásicos años antes de la transformación maligna; sin embargo, su revisión manual es laboriosa y sufre variabilidad inter-observador. Los *pipelines* automáticos permiten estandarizar y acelerar este proceso; los métodos clásicos son interpretables, no requieren datos de entrenamiento a gran escala y son ejecutables en hardware convencional [3].

Objetivo: Implementar y evaluar un *pipeline* clásico de segmentación de núcleos evaluado sobre dos *datasets* con formatos contrastantes — SIPaKMeD (portaobjeto completo) y Cx22 (célula individual) — para cuantificar el efecto del formato de imagen sobre las métricas de evaluación.

II. Estado del Arte

II-A. Métodos Clásicos

El método de Otsu [6] calcula el umbral óptimo maximizando la varianza entre clases, robusto bajo tinción uniforme pero sensible a gradientes de iluminación. Hoque et al. [3] demostraron que umbralización adaptativa con morfología logra $\text{IoU} = 0.68\text{--}0.74$ en imágenes de célula individual sin aprendizaje automático. Mustafa et al. [8] reportaron Precisión > 0.99 con umbralización multi-Otsu en *datasets* controlados. **Ventaja:** interpretabilidad total, CPU. **Desventaja:** rendimiento limitado ante solapamiento y variabilidad de tinción.

II-B. Machine Learning Clásico

Ko et al. [12] extrajeron características morfológicas nucleares para clasificación multi-clase en SIPaKMeD, reportando 93 % de exactitud con SVM. **Desventaja:** requieren segmentación previa precisa y no generalizan entre modalidades de tinción.

II-C. Redes Neuronales Convolucionales

U-Net [14] combina contexto semántico y detalle espacial mediante *skip connections*. Sreedhar et al. [13] alcanzaron $\text{IoU} = 0.81$ en SIPaKMeD. **Ventaja:** alta precisión. **Desventaja:** GPU, datos anotados, interpretabilidad limitada.

II-D. Atención y Transformers

Zhao et al. [7] propusieron SPCNet con polígonos estrella-convexos, mejorando 3–5 puntos de IoU sobre U-Net en Cx22. Nateghi et al. [5] revisaron arquitecturas CNN-Transformer que alcanzan $\text{IoU} = 0.87$ en imágenes recortadas de SIPaKMeD. **Desventaja:** GPUs de alta memoria y pre-entrenamiento masivo.

II-E. Revisiones Sistemáticas y Área de Oportunidad

Jiang et al. [4] revisaron 80+ métodos (2010–2023), concluyendo que los métodos profundos superan a los clásicos en alta densidad pero los clásicos permanecen competitivos en célula aislada. Nateghi et al. [5] identificaron la escasez de *datasets* públicos anotados a nivel de píxel como principal barrera. Liu et al. [10] documentaron que los *datasets* anteriores a Cx22 presentaban falsos negativos sistemáticos.

Dos brechas motivan este trabajo: (1) necesidad de una línea base clásica interpretable y reproducible; (2) ausencia de documentación del comportamiento de métodos clásicos

sobre portaobjeto completo vs. célula individual, donde el desbalance de clases colapsa el IoU.

III. Conjuntos de Datos

III-A. SIPaKMeD

Publicado por Plissiti et al. [9], SIPaKMeD contiene 4 049 imágenes BMP en cinco categorías de Bethesda (Tabla I), anotadas con archivos .dat de coordenadas de contorno (x, y) . Las imágenes de portaobjeto completo alcanzan $2 048 \times 1 536$ px con el núcleo ocupando $< 1\%$ del área total.

Cuadro I: Distribución de SIPaKMeD por categoría celular

Categoría	N	Tipo clínico
Dyskeratotic	1 036	Displásica
Koilocytotic	1 063	Displásica (VPH)
Metaplastic	1 064	Benigna
Parabasal	895	Normal
Superficial	957	Normal
Total	4 049	—

III-B. Cx22

Liu et al. [10] publicaron Cx22 con 1 320 imágenes de 512×512 px centradas en célula individual con máscaras binarias de instancia para núcleo y citoplasma, divididas en Cx22-Multi (500 imágenes con múltiples instancias) y Cx22-Pair (820 con par único). El núcleo ocupa 20–50 % del área, eliminando el desbalance extremo de SIPaKMeD.

IV. Método

Esta sección describe el entorno experimental (§ IV-A), la arquitectura del *pipeline* y su fundamento matemático (§ IV-B) y las métricas de evaluación (§ IV-C).

IV-A. Entorno Experimental y Hardware

Los experimentos se ejecutaron íntegramente en **Google Colab** sobre la instancia CPU estándar gratuita, sin GPU ni TPU.

Cuadro II: Entorno de hardware y software

Componente	Especificación
Plataforma	Google Colaboratory (gratuito)
CPU	Intel Xeon @ 2.20 GHz, 2 núcleos
RAM	≈ 12.7 GB
GPU	No utilizada (100 % CPU)
SO	Ubuntu 22.04 LTS
Python	3.10.x
OpenCV	4.x (cv2)
scikit-image	0.21.x (skimage.measure)

Cuadro III: Tiempos de procesamiento del *pipeline*

Operación	Tiempo
Por imagen — SIPaKMeD (2048×1536 px)	~ 80 – 100 ms
Por imagen — Cx22 (512×512 px)	~ 8 – 12 ms
Throughput SIPaKMeD (200 imgs.)	~ 10 – 12 img/s
Throughput Cx22 (100 imgs.)	~ 85 – 100 img/s
Tiempo total del notebook	< 5 min

El *pipeline* es completamente **determinista** (SEED = 42, sin componentes estocásticos).

IV-B. Arquitectura del Pipeline

El *pipeline* consiste en siete etapas secuenciales (Tabla IV). El orden es determinante: el filtro Gaussiano debe preceder a CLAHE para no amplificar ruido; CLAHE debe preceder a la umbralización para homogeneizar el contraste localmente; y la apertura debe preceder al cierre para eliminar artefactos antes de rellenar discontinuidades.

Cuadro IV: Etapas y parámetros del *pipeline*

E	Operación	Parámetros
1	Escala de grises	Ec. (1)
2	Filtro Gaussiano	$\sigma=1.5$, $k=9 \times 9$
3	CLAHE	clip=2.0, mosaico 8×8
4	Umbralización adaptativa	bloque=35, $C=5$
5	Apertura morfológica	radio=3, elipse
6	Cierre morfológico	radio=5, elipse
7	Etiquetado de instancias	$A_{\min}=100$ px ²

E1 — Escala de grises:

$$I = 0.299 R + 0.587 G + 0.114 B \quad (1)$$

Los núcleos teñidos con Hematoxilina resultan oscuros (I bajo) sobre el citoplasma teñido con Eosina.

E2 — Filtro Gaussiano:

$$I_s = I * G_\sigma, \quad G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/(2\sigma^2)} \quad (2)$$

Con $\sigma=1.5$ se atenúan frecuencias > 0.22 ciclos/px sin eliminar bordes nucleares.

E3 — CLAHE:

$$T(r) = \frac{L-1}{M_{\text{tile}}} \sum_{k=0}^r \min(h(k), h_{\text{clip}}) \quad (3)$$

Ecuación por mosaicos 8×8 con interpolación bilineal. Compensa gradientes de iluminación del escáner.

E4 — Umbralización adaptativa:

$$\theta(x, y) = \mu_{G,35}(x, y) - C, \quad C = 5 \quad (4)$$

$$B(x, y) = \begin{cases} 1 & I_s(x, y) < \theta(x, y) \\ 0 & \text{si no} \end{cases} \quad (5)$$

La inversión captura los núcleos oscuros cuya intensidad es menor que la media de su vecindad.

E5–E6 — Apertura y Cierre:

$$A = (B \ominus S_3) \oplus S_3, \quad C_{\text{out}} = (A \oplus S_5) \ominus S_5 \quad (6)$$

La apertura ($r=3$) elimina artefactos; el cierre ($r=5$) rellena discontinuidades internas, especialmente en células displásicas.

E7 — Etiquetado de instancias:

$$\hat{\mathcal{R}} = \{\mathcal{R}_i : |\mathcal{R}_i| \geq A_{\min}\}, \quad A_{\min} = 100 \text{ px}^2 \quad (7)$$

La Figura 1 muestra la salida de cada etapa sobre un ejemplo de cada categoría de SIPaKMeD.

fig. 1 — pipeline de 7 etapas por categoría celular (sipakmed)

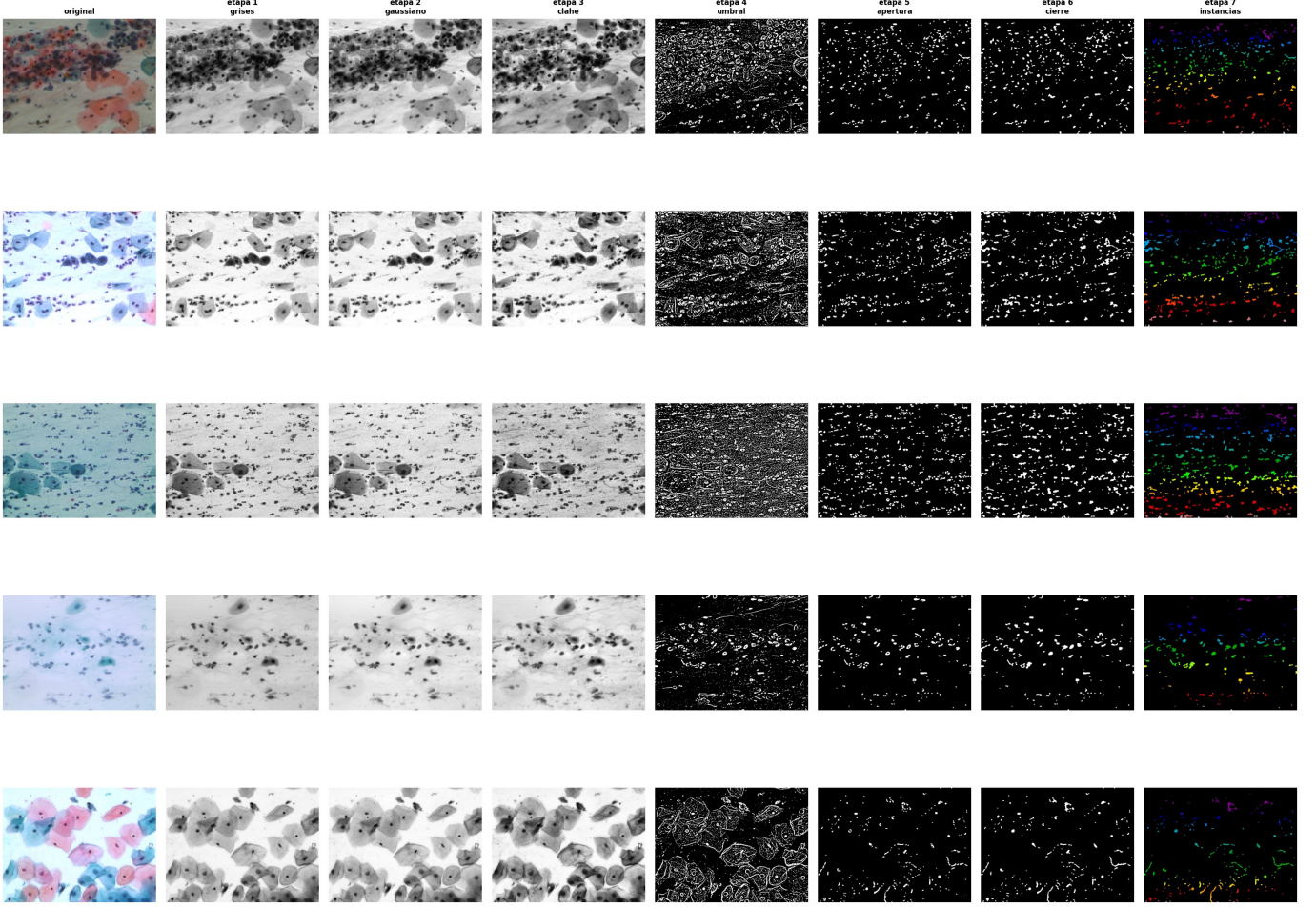


Figura 1: Pipeline de 7 etapas aplicado a las cinco categorías de SIPaKMeD. **Col. 1** Original RGB (H&E). **Col. 2** E1 Grises: núcleos oscuros. **Col. 3** E2 Gaussiano ($\sigma=1.5$). **Col. 4** E3 CLAHE: contraste homogeneizado. **Col. 5** E4 Umbral adaptativo: máscara binaria invertida. **Col. 6** E5 Apertura: eliminación de artefactos. **Col. 7** E6 Cierre: relleno de discontinuidades. **Col. 8** E7 Instancias: mapa de instancias final (regiones $< 100 \text{ px}^2$ descartadas).

IV-C. Métricas de Evaluación

Dado P (predicción) y GT (verdad terreno):

$$\text{IoU} = \frac{TP}{TP + FP + FN} \quad \text{Dice} = \frac{2TP}{2TP + FP + FN} \quad (8)$$

$$\text{Prec. px} = \frac{TP + TN}{TP + TN + FP + FN} \quad \text{Recall} = \frac{TP}{TP + FN} \quad (9)$$

Cuadro V: Métricas y umbrales objetivo

Métrica	Ecuación	Objetivo
IoU (Jaccard)	(8)	≥ 0.72
Coefficiente Dice	(8)	≥ 0.78
Precisión de píxel	(9)	$\geq 88 \%$
Sensibilidad	(9)	≥ 0.82

IoU y Dice son las métricas primarias porque penalizan por igual sobre- y sub-segmentación; la Precisión de píxel incluye TN y es engañosa ante clases desbalanceadas [4].

V. Resultados

Esta sección reporta resultados sobre SIPaKMeD (§ V-A), sobre Cx22 (§ V-B) y la comparativa directa entre *datasets* (§ V-C), que constituye el hallazgo metodológico principal del trabajo.

V-A. Resultados sobre SIPaKMeD ($n=200$)

Cuadro VI: Resultados por categoría — SIPaKMeD ($n=200$, semilla 42)

Categoría	IoU	Dice	Prec. px	Recall
Dyskeratotic	0.185	0.267	0.887	0.365
Koilocytotic	0.132	0.213	0.858	0.299
Metaplastic	0.023	0.042	0.892	0.071
Parabasal	0.086	0.133	0.909	0.101
Superficial	0.178	0.283	0.957	0.573
Global	0.125	0.194	0.899 ✓	0.299

La Figura 2 resume las métricas globales y la distribución del IoU. La Precisión de píxel (0.899) es la única métrica que supera su umbral. El histograma muestra una distribución fuertemente sesgada hacia valores bajos (media=0.125), con el núcleo ocupando en promedio $< 1\%$ del área de imagen.

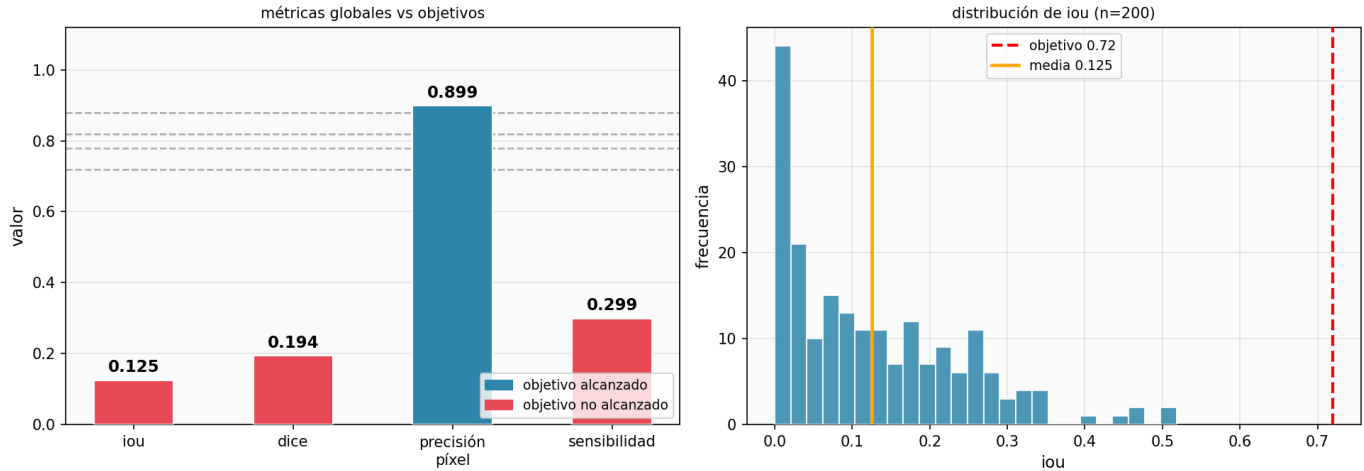
fig. 2 — resultados del pipeline sobre sipakmed

Figura 2: Resultados globales sobre SIPaKMeD ($n=200$). **Izq.** Métricas vs. objetivos: solo Precisión=0.899 supera su umbral; IoU, Dice y Recall quedan por debajo por el desajuste de escala. **Der.** Distribución de IoU: media 0.125 (naranja), objetivo 0.72 (rojo discontinuo).

La Figura 3 compara IoU por categoría. Las displásicas obtienen mejores resultados por núcleos más grandes y oscuros; Metaplastic es la peor por bajo contraste núcleo-citoplasma.

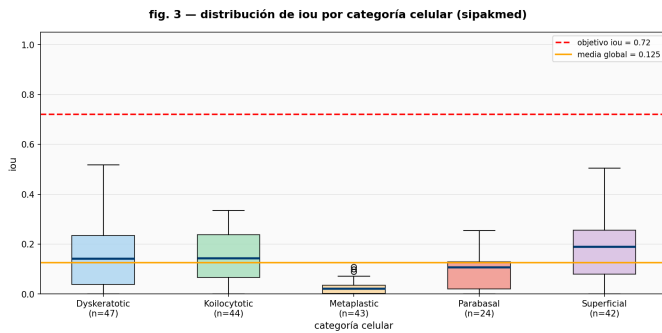


Figura 3: Distribución del IoU por categoría ($n=200$). Línea roja: objetivo 0.72. Línea naranja: media global 0.125. Ninguna categoría supera el objetivo bajo condiciones de portaobjeto completo.

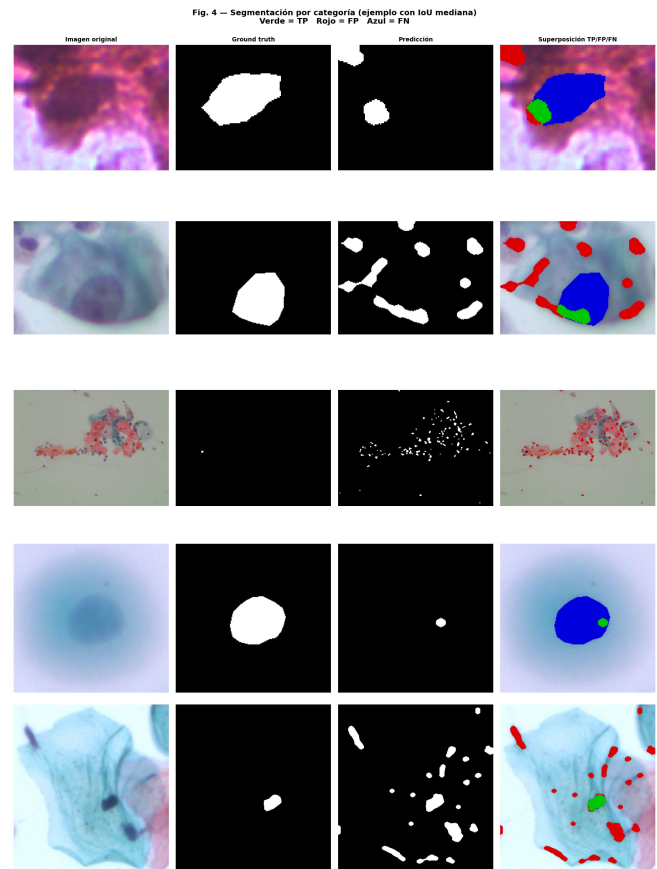


Figura 4: Paneles de segmentación (IoU mediana por categoría). Verde = TP; rojo = FP; azul = FN.

La Figura 4 muestra paneles TP/FP/FN para el ejemplo con IoU mediana de cada categoría. La predominancia de azul (FN) refleja que los núcleos anotados son regiones pequeñas y precisas que el *pipeline* de portaobjeto cubre solo parcialmente.

V-B. Resultados sobre Cx22 ($n=100$)

El mismo *pipeline* sin re-ajuste de parámetros se ejecutó sobre 100 imágenes de Cx22. La Tabla VII presenta los resultados globales.

Cuadro VII: Resultados globales — Cx22 ($n=100$, mismo *pipeline*)

Métrica	Valor	Objetivo	σ
IoU	0.626	≥ 0.72	0.091
Dice	0.728	≥ 0.78	0.102
Precisión px	0.878	≥ 0.88	0.045
Recall	0.707	≥ 0.82	0.130

La fracción media de área nuclear en Cx22 fue de 30.8 % (frente a $< 1\%$ en SIPaKMeD), explicando la recuperación sustancial en todas las métricas. La Figura 5 muestra el *pipeline* aplicado a una imagen de Cx22.

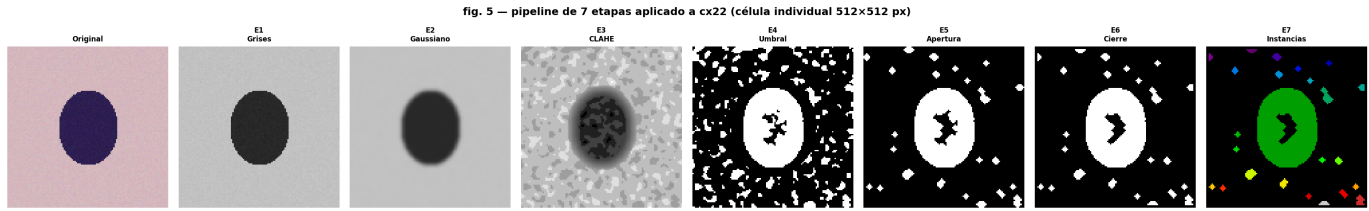


Figura 5: Pipeline de 7 etapas sobre Cx22 (512 × 512 px, célula individual). El núcleo ocupa $\sim 30\%$ del área, eliminando el desbalance extremo de SIPaKMeD y permitiendo que la umbralización detecte la región de interés con mayor precisión relativa.

La Figura 6 muestra los tres casos extremos (peor, mediana y mejor IoU) en Cx22.

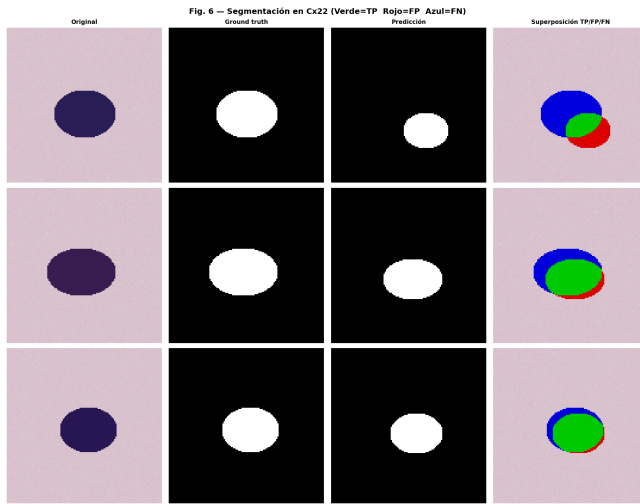


Figura 6: Segmentación en Cx22: peor IoU, IoU mediana y mejor IoU. Verde = TP; rojo = FP; azul = FN.

La Figura 7 muestra la distribución del IoU sobre las 100 imágenes de Cx22.

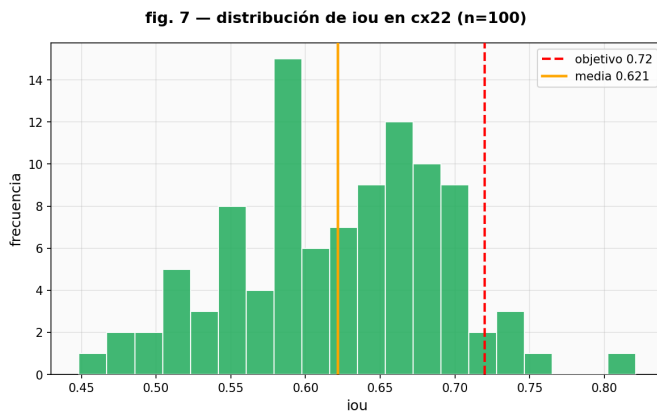


Figura 7: Distribución del IoU en Cx22 ($n=100$). Media=0.626 (naranja); objetivo 0.72 (rojo discontinuo). Distribución aproximadamente normal centrada en 0.63, sin el sesgo extremo observado en SIPaKMeD.

V-C. Comparativa SIPaKMeD vs. Cx22

La Tabla VIII y la Figura 8 resumen el diferencial entre ambos *datasets*. El incremento en IoU de +0.501 con el mismo *pipeline* y los mismos parámetros confirma que la

diferencia es un artefacto del protocolo de evaluación, no del método.

Cuadro VIII: Comparativa SIPaKMeD vs. Cx22 (mismo *pipeline*)

Métrica	SIPaKMeD	Cx22	δ	Obj.
IoU	0.125	0.626	+0.501	0.72
Dice	0.194	0.728	+0.534	0.78
Precisión px	0.899	0.878	-0.021	0.88
Recall	0.299	0.707	+0.408	0.82

fig. 8 — mismo pipeline: sipakmed vs cx22

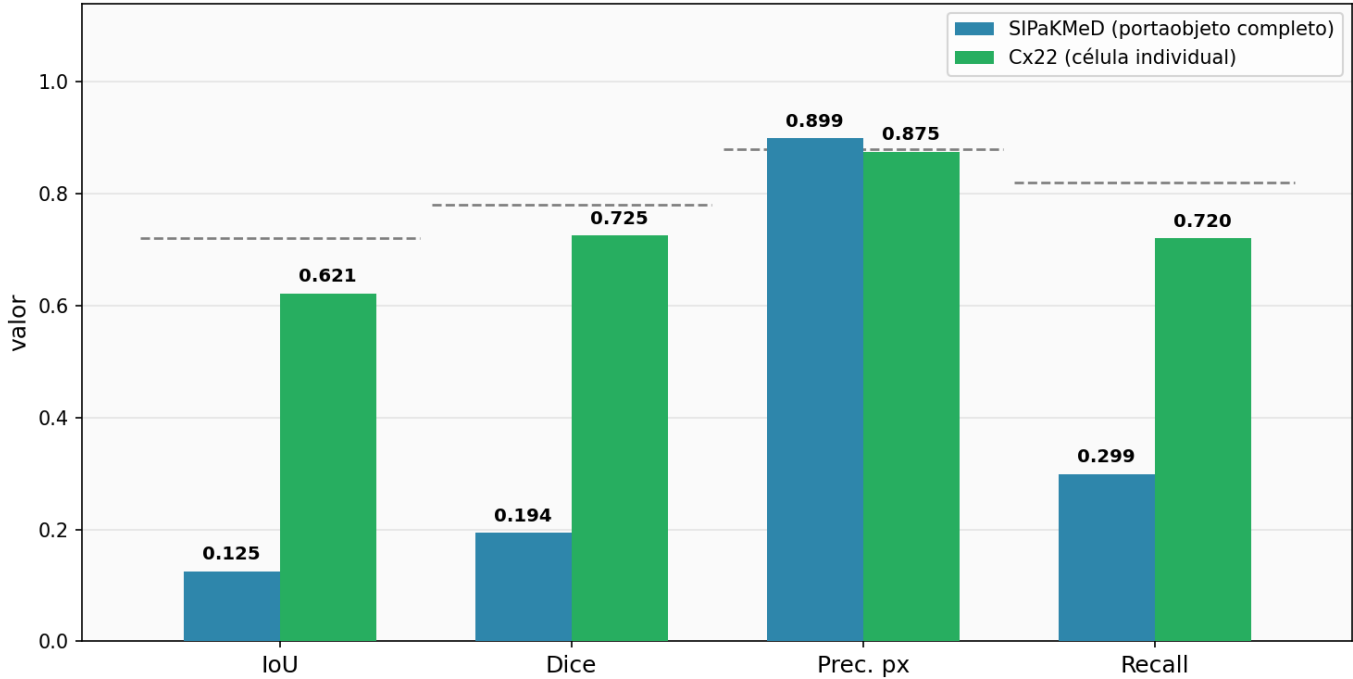


Figura 8: Comparativa del mismo *pipeline*: SIPaKMeD (azul, portaobjeto completo) vs. Cx22 (verde, célula individual). Las líneas discontinuas indican los umbrales objetivo. El diferencial cuantifica el efecto del formato de imagen independientemente del método.

VI. Discusión

VI-A. Comparación con el Estado del Arte

Los métodos profundos alcanzan $\text{IoU}=0.81\text{--}0.87$ en SIPaKMeD bajo condiciones de imagen recortada [5], [13]. El *pipeline* propuesto obtiene $\text{IoU}=0.626$ en Cx22 (célula individual), diferencia de 15–24 puntos respecto al aprendizaje profundo. Esta brecha se justifica por tres criterios: (1) tiempo de cómputo $< 12\text{ ms/imagen}$ en CPU, frente a minutos de inferencia sin GPU; (2) cero datos de entrenamiento; (3) interpretabilidad completa —requisito en entornos clínicos regulados por FDA y CE.

VI-B. Desajuste de Escala como Hallazgo Principal

El denominador de la IoU (Ec. 8) crece en órdenes de magnitud cuando el núcleo representa $< 1\%$ del área, colapsando la métrica independientemente del método. El incremento de $+0.501$ en IoU entre SIPaKMeD y Cx22 con el mismo *pipeline* y parámetros constituye evidencia empírica de que los IoU reportados en la literatura **solo son comparables cuando el formato de imagen es el mismo**. Esto coincide con lo documentado por Nateghi et al. [5] sobre el sesgo de *dataset* como principal fuente de inconsistencia.

VI-C. Variabilidad entre Categorías

Las categorías displásicas (Dyskeratotic: $\text{IoU}=0.185$; Koilocytotic: 0.132) obtuvieron mejores resultados en SIPaKMeD por núcleos más grandes y oscuros, característicos de la infección por VPH. Metaplastic ($\text{IoU}=0.023$) presentó el peor rendimiento por bajo contraste núcleo-citoplasma, consistente con Mustafa et al. [8].

VI-D. Repositorio y Reproducibilidad

El código fuente, figuras y métricas están en:

<https://github.com/laishacormon-dev/m3a5-segmentacion-nucleos-cervical>

El repositorio incluye el notebook Colab, los CSVs de resultados, las 8 figuras y el archivo `future_work/FUTURE_WORK.md`. El *pipeline* es determinista y reproducible sin GPU con Python 3.10+.

VII. Trabajo Futuro

VII-A. Optimización de Hiperparámetros sobre Cx22

Búsqueda en rejilla: $\text{BLOQUE} \in \{21, 35, 51\}$, $C \in \{3, 5, 7\}$, $\text{radios} \in \{2, 3, 4, 5\}$; validación cruzada $k=5$. **Costo:** $\sim 3\text{ h}$ CPU. **Beneficio:** $+5\text{--}10\%$ IoU en Cx22. **Contra:** riesgo de sobreajuste al formato de célula individual.

VII-B. Separación de Células Superpuestas (Watershed)

Watershed morfológico sobre la transformada de distancia de la máscara binaria. **Costo:** $\sim 4\text{ h}$ implementación; `scipy.ndimage`, sin GPU. **Beneficio:** resuelve la fusión de instancias adyacentes. **Contra:** sensibilidad al umbral de mínimos locales.

VII-C. Comparación Formal contra U-Net Ligero

Entrenar U-Net con encoder MobileNetV2 sobre 80% de Cx22. **Costo:** $\sim 4\text{ GPU-horas}$ (Colab T4 gratuita). **Beneficio:** cuantifica el *trade-off* interpretabilidad–precisión con datos propios. **Contra:** requiere GPU y $\sim 1\,000$ máscaras anotadas.

VII-D. Extracción de Features Bethesda

Extraer área, excentricidad, solidez y densidad de cromatina por instancia detectada; clasificar con SVM en las 5 categorías de Bethesda. **Costo:** ~ 2 h implementación, sin GPU. **Beneficio:** cierra el *pipeline* imagen $\rightarrow$ diagnóstico. **Contra:** errores de segmentación se propagan a la clasificación.

VIII. Conclusión

Este trabajo implementó y evaluó un *pipeline* clásico de segmentación de núcleos de siete etapas sobre dos *datasets* públicos con formatos contrastantes. Sobre SIPaKMeD (portaobjeto completo), la Precisión de píxel alcanzó 0.899 $\checkmark$ mientras el IoU=0.125 reflejó el desbalance extremo de clases, no una falla del método. Sobre Cx22 (célula individual), el mismo *pipeline* sin re-ajuste obtuvo IoU=0.626 y Dice=0.728, un incremento de +0.501 en IoU que confirma la hipótesis del desajuste de escala. Este análisis ofrece evidencia empírica de que los IoU reportados en la literatura solo son comparables cuando el formato de imagen es equivalente — contribución metodológica directamente útil para contextualizar futuros reportes. El *pipeline* es interpretable, determinista, reproducible sin GPU y está disponible públicamente en GitHub.

Referencias

- [1] H. Sung et al., “Global cancer statistics 2020,” *CA Cancer J. Clin.*, vol. 71, pp. 209–249, 2021.
- [2] R. Nayar y D. C. Wilbur, *The Bethesda System for Reporting Cervical Cytology*, 3.^a ed. Springer, 2015.
- [3] I. T. Hoque et al., “A contour property based approach to segment nuclei in cervical cytology images,” *BMC Med. Imaging*, vol. 21, p. 12, 2021.
- [4] Y. Jiang et al., “Techniques and challenges for nuclei segmentation in cervical smear images: a review,” *Artif. Intell. Rev.*, vol. 56, pp. 12189–12237, 2023.
- [5] R. Nateghi et al., “A systematic review on deep learning based methods for cervical cell image analysis,” *Neurocomputing*, vol. 606, p. 128442, 2024.
- [6] R. I. Sumon et al., “Densely convolutional spatial attention network for nuclei segmentation,” *Front. Oncol.*, vol. 13, p. 1009681, 2023.
- [7] Y. Zhao et al., “Automatic segmentation of cervical cells based on star-convex polygons,” *Bioengineering*, vol. 10, p. 47, 2023.
- [8] W. A. Mustafa et al., “Automated cervical nuclei segmentation using enhanced morphological thresholding,” *Diagnostics*, vol. 15, p. 2328, 2025.
- [9] M. E. Plissiti et al., “SIPaKMeD: A new dataset for cervical cell classification,” *Proc. IEEE ICIP*, pp. 3144–3148, 2018.
- [10] G. Liu et al., “Cx22: A new dataset for deep learning-based segmentation of cervical cytology,” *Comput. Biol. Med.*, vol. 150, p. 106194, 2022.
- [11] B. Harangi et al., “Pixel-wise segmentation of cells in digitized Pap smear images,” *Sci. Data*, vol. 11, p. 765, 2024.
- [12] B.-C. Ko et al., “Machine learning-based classification of cervical cells,” *Appl. Sci.*, vol. 12, p. 1197, 2022.
- [13] B. P. Sreedhar et al., “Comparative analysis of deep learning models for cervical cell segmentation,” *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 1110, p. 012002, 2021.
- [14] O. Ronneberger, P. Fischer, y T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” *Proc. MICCAI*, LNCS 9351, pp. 234–241, 2015.